

Steps to Build a Simple "Add Two Numbers" Function

Planned and explained via Claude Code Plan Mode

1. Define the requirement

State precisely what the function must do: take two numbers, return their sum.

Purpose: Prevents scope creep and ambiguity later (should it handle strings? more than two arguments?). A clear, narrow requirement is the contract everything else gets checked against.

2. Decide the function signature

```
function addNumbers(a: number, b: number): number
```

Purpose: The signature is the function's public contract. Explicit types and a descriptive name make it self-documenting and catch misuse at compile time instead of runtime.

3. Write the implementation (happy path)

```
function addNumbers(a: number, b: number): number {  
    return a + b;  
}
```

Purpose: The minimal logic that satisfies the requirement — nothing here needs abstracting or configuring.

4. Consider edge cases

`NaN`, very large numbers (floating-point precision), negatives.

Purpose: Even simple functions have boundary behavior worth being deliberate about. Here, TypeScript's type system already rules out non-numeric input, so "no runtime guard needed" is itself a conscious design decision, not an oversight.

5. Write a test

```
test('addNumbers adds two numbers', () => {  
  expect(addNumbers(2, 3)).toBe(5);  
});
```

Purpose: Proves the function does what it claims and protects against future regressions.

6. Validate the design

Does it do one thing? Is the signature strict (no `any`)? Would a junior developer understand it immediately?

Purpose: The review pass — catches over-engineering (needless options) or under-engineering (missing types) before calling it done.

7. Run and verify

Run the test, confirm it passes; optionally exercise it manually (`node -e "console.log(addNumbers(2,3))" → 5`).

Purpose: Closes the loop between intent (step 1) and actual, proven result.

No files in the repository were changed — this was a standalone explanatory walkthrough.

Generated by Claude Code